IU2541230781 – CPP71 – SUBMISSION TEMPLATE

Project Name: Vehicle Parking & Tracking

Enrollment No.: IU2541230781

Student Name: Hitendra Singh Solanki

Date of Submission: 12/10/2026

1. Project Summary

Problem Statement: Manual parking records can be difficult to maintain, search and update. This project provides a menu-driven C++ application to manage vehicle and owner records, support parking entry and exit, and save records between sessions.

Objectives: Manage vehicle/owner records; perform parking and exit transactions; display reports; demonstrate OOP concepts, dynamic memory, validation and exception handling; and persist records using file handling.

Key Features: Add, display, search, update and delete records; park and exit vehicles; view active parking and reports; save/load records using parking_data.txt; menu validation; and automatic save on exit.

2. Class Summary

Class	Purpose / Responsibility	Key Data Members	Key Member Functions
Person	Base class for common person information	name	constructor; virtual displayDetails(); virtual destructor
Owner	Stores vehicle owner information	ownerId, name inherited from Person, phone	constructor; displayDetails() override; getters/setters
Vehicle	Stores a vehicle record	vehicleNo, type, ownerName	constructors; display(); getters/setters
Slot	Represents a parking slot and its availability	slotId, type, occupied	constructors; reserve(); release(); display()
ParkingRecord	Tracks a parking transaction and fee	recordId, vehicleNo, slotId, entryTime, exitTime, fee, active	constructors; calculateFee(); closeRecord(); display()
ParkingSystem	Coordinates records, parking operations and menu	dynamic arrays/pointers for vehicles, owners, slots, records; counts	add/display/search/update/delete; park/exit; report; save/load; run
FileManager	Loads and saves application data	file name and file streams	static save(); static load()

3. Project Flow

Actual project flow: The user selects an operation from the main menu; ParkingSystem validates and processes it using the entity classes; FileManager loads or saves records in parking_data.txt.

USER → MAIN MENU → ParkingSystem / Entity Classes → FileManager → parking_data.txt

4. UML Deliverables

Diagram	File Name	Completed
Use Case Diagram	Use_Case_Diagram.png	☒
Class Diagram	Class_Diagram.png	☒
Sequence Diagram 1	Sequence_Diagram_1_Main_Parking_Transaction.png	☒
Sequence Diagram 2	Sequence_Diagram_2_Search_Record.png	☒

5. C++ Concepts Demonstrated

Concept	Where Used in My Project	Explanation
Encapsulation	Private data in Vehicle, Owner, Slot, ParkingRecord and ParkingSystem	Data is kept with the operations that manage it and accessed through public member functions.

Concept	Where Used in My Project	Explanation
Constructor	Constructors in Person, Owner, Vehicle, Slot, ParkingRecord and ParkingSystem	Default and parameterised constructors initialise objects with suitable starting values.
Destructor	ParkingSystem destructor; virtual destructor in Person	Dynamically allocated arrays are released with delete[]; virtual destructor supports safe base-class cleanup.
Inheritance	Owner : public Person	Owner reuses the common name field and overrides displayDetails().
Runtime Polymorphism	virtual Person::displayDetails() and Owner override	The overridden virtual function allows derived-class behaviour through a base-class interface.
Dynamic Memory	Vehicle*, Owner*, Slot* and ParkingRecord* arrays in ParkingSystem	The arrays are allocated with new[] and released by the destructor with delete[].
File Handling	FileManager::save() and FileManager::load()	The system writes and reads parking records in parking_data.txt.
Exception Handling	try/catch in the program menu and main()	Exceptions are caught to show an error message and prevent an unhandled failure.

6. Testing

Test ID	Operation	Input	Expected Output	Actual Output	Pass/Fail
T01	Add Record	Valid vehicle and owner details	Record accepted and added	Record added successfully	Pass
T02	Display Records	Select option 2	Stored records displayed	Vehicle/owner records displayed	Pass
T03	Search Record	Search an existing vehicle/owner	Matching record found	Matching record displayed	Pass
T04	Search Record	Search a record that does not exist	Not-found message shown	Not-found message shown	Pass
T05	Update Record	Choose an existing record and change details	Updated details stored	Updated details shown after operation	Pass
T06	Delete Record	Choose an existing record	Record deleted	Record deletion confirmed	Pass
T07	Park Vehicle	Select a registered vehicle with a compatible free slot	Vehicle parked and slot allocated	Parking transaction started	Pass
T08	Park Vehicle	Try to park an unregistered vehicle	Operation rejected with guidance	Unregistered vehicle rejected	Pass
T09	Exit Vehicle	Exit a vehicle currently parked	Transaction closed and fee shown	Exit processed and fee displayed	Pass
T10	Active Parking	Open active parking view	Currently parked vehicles displayed	Active parking list displayed	Pass
T11	Report	Select report option	Report summary displayed	Report summary displayed	Pass
T12	Save Data	Use Save option	Data written to file	Save operation completed	Pass
T13	Load Data	Use Load option	Saved records loaded	Records loaded from file	Pass
T14	Persistence	Save, exit and restart the program	Previously saved records available	Saved record remained available after restart	Pass
T15	Input Validation	Enter invalid menu/input value	Invalid input handled safely	Invalid input was handled	Pass

7. Deployment

Compiler / IDE Used: g++ (C++17) and Visual Studio Code

Compilation Command: g++ -std=c++17 main.cpp -o VehicleParking.exe

Executable Name: VehicleParking.exe

Data File(s): parking_data.txt

8. Student Declaration

I understand the submitted project and can explain its classes, data members, member functions, file handling and UML diagrams.

Student Signature: Hitendra Singh Solanki